



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

RASA FRAMEWORK

From Intents to Intelligence:
A Hands-on Journey with Rasa CALM

ONE LOVE. ONE FUTURE.

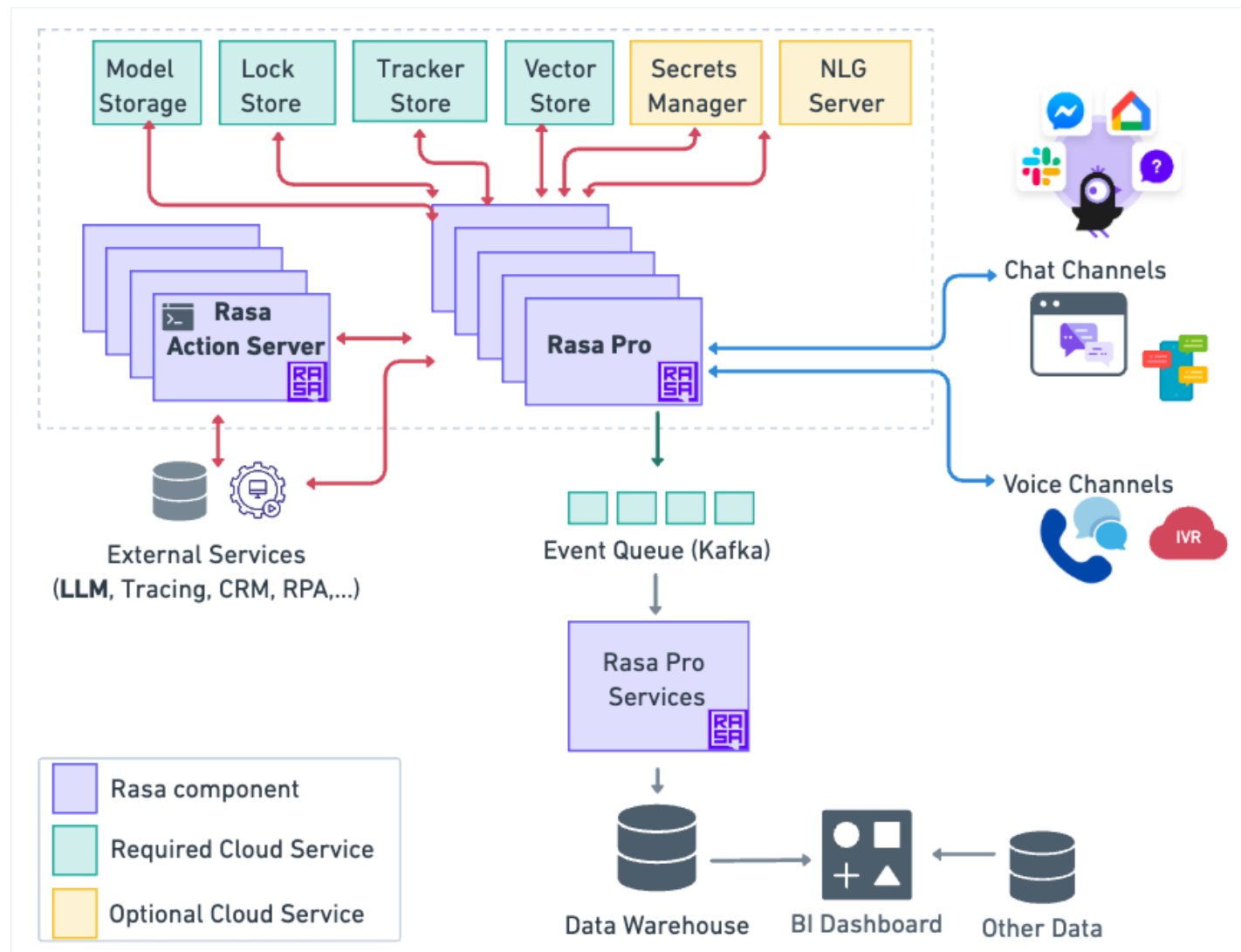
Learning Objectives

- Get introduced to **Rasa** and understand its **key advantages**
- Comprehend the **architecture** of Rasa and its **main components**
- **Install and configure** a development environment for building assistants with Rasa
- Build and test a **simple chatbot** through hands-on practice

Why Use Rasa?

- Open-source framework for **conversational AI**
- Focus on:
 - **Customization**
 - **Privacy & control**
 - **Production-grade deployment**
- Widely used in:
 - Banking & Insurance
 - Healthcare
 - E-commerce
- Rasa CALM: New flow-based architecture optimized for LLM integration

Overview of Rasa's Architecture



Project Introduction: AI Travel Booking Assistant

AI Travel Booking Assistant is a hands-on project built using Rasa CALM, aiming to develop a conversational assistant that supports users in the following tasks:

- Searching for available flight information
- Booking flight tickets to specific destinations
- Checking details of previously booked flights
- Cancelling flights when cancellation is permitted

This project showcases how to apply key features of Rasa CALM, including:

- Flow-based dialogue management for handling conversation logic
- Custom actions for implementing backend operations
- Integration with large language models (LLMs) to enable natural and context-aware interactions

This practical session will guide participants through designing, building, and testing a complete travel booking assistant.

Project Objectives

By the end of this session, participants will be able to:

- Understand how to design a conversational assistant using flow-based dialogue
- Learn how to use Rasa CALM and integrate it with an LLM
- Build and execute custom actions to handle flight booking logic
- Connect Rasa to external data sources (e.g., CSV files or a mock database)

The assistant will guide users through common travel-related tasks, including:

- Searching for available flight information
- Booking flight tickets to selected destinations
- Checking details of existing bookings
- Cancelling bookings when cancellation is allowed

System Architecture

This project is built on a modular architecture that includes the following key components:

- **Rasa Core & NLU:** Manages dialogue flow and understands user intents and entities
- **Rasa Actions:** Handles custom logic such as retrieving flight data, booking tickets, or processing cancellations
- **CSV Files / Mock Database:** Simulates backend storage for available flights and bookings
- **LLM Integration via Rasa CALM:** Enhances conversation quality with natural language understanding and generation using a large language model

Install and Setup

1. Setup environment

- Download [miniconda](#)
- Create new rasa-env using python 3.9:
`conda create --name rasa-env python=3.9`
- Activate the virtual environment
`conda activate rasa-env`
Deactivate an activate environment
`conda deactivate`
- Install rasa-pro with:
`pip install rasa-pro`

2. Obtain and Set Up the Rasa Pro License

- [Request a License Key](#)

- For Temporary Use:

- Set the License Key as an Environment Variable:
window powershell

```
$env:RASA_PRO_LICENSE="your_license_key"
```

macos/linux:

```
export RASA_PRO_LICENSE=your_license_key
```

- For Persistent Use:

window powershell

```
[System.Environment]::SetEnvironmentVariable('RASA_PRO_LICENSE',  
'your_license_key', 'User')
```

macos/linux:

```
echo "export RASA_PRO_LICENSE=your_license_key" >> ~/.bashrc  
source ~/.bashrc
```

3. Initializing a New Rasa Project

- Create new directory named:

```
mkdir rasa_project
```

- Navigate into rasa_project folder:

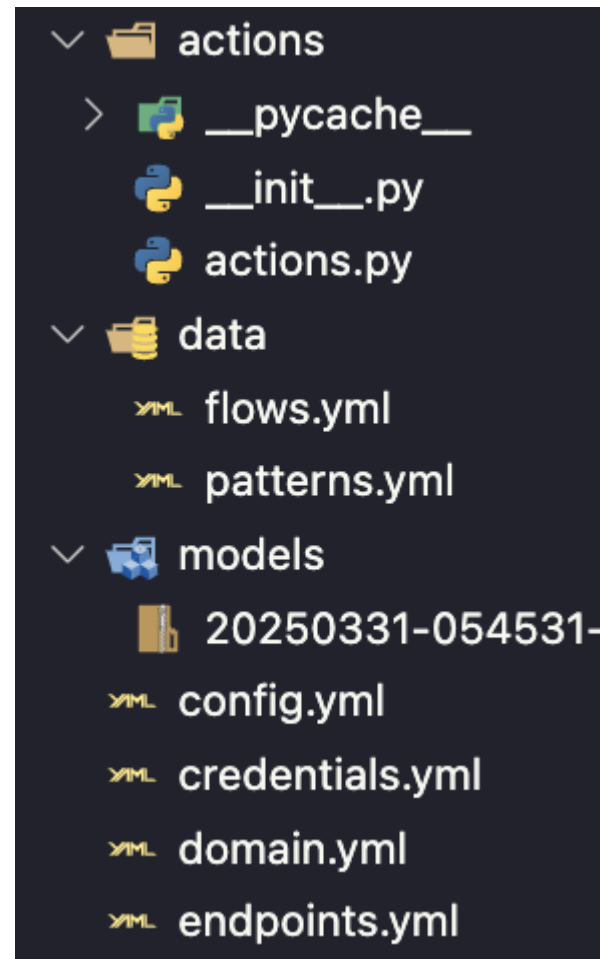
```
cd rasa_project
```

- Initialize the Rasa project:
 - `rasa init`: create an NLU-based assistant
 - `rasa init --template calm`: create a CALM assistant

Project Directory Structure

Key Files and Directories:

- **data/**: Contains training data for NLU and Core.
- **models/**: Stores trained models.
- **domain.yml**: Defines intents, entities, slots, responses, and actions
- **config.yml**: Configures the processing pipeline and policies.
- **endpoints.yml**: Sets up endpoints for action servers and trackers.
- **flows.yml**: Define the business logic rules of your assistant.
- **actions.py**: Write the actual code of the custom actions.



What Are Flows?

- Flows are structured sequences of steps that encapsulate the business logic required to achieve specific user goals, such as blocking a credit card, changing an address, or adding a payee.
- By isolating business logic from the broader conversation, Flows offer:
 - **Clarity:** Each Flow is dedicated to a single task or outcome.
 - **Reusability:** Flow logic can be invoked from other Flows or initiated independently.
 - **Maintainability:** Modifications to Flow logic can be made without impacting the overall conversation design.

Defining a Flow in `flows.yml`:

- Each Flow is specified using YAML syntax with key attributes:
 - **Flow ID:** A unique identifier for the Flow.
 - **Name (optional):** A human-readable name for the Flow.
 - **Description:** A brief explanation of the Flow's purpose.
 - **Steps:** A list of actions or information requests within the Flow.
 - **NLU Trigger property:** used to add [intents that can start the Flow](#).
 - **If property:** A field is used to add [flow guards](#).

Basic Structure of a Flow

Example of a Simple Flow:

- A Flow to greet the user:

```
flows:  
  greet_user:  
    description: "Flow to greet the user"  
    steps:  
      - action: utter_greet
```


Key Components of a Flow

Collect Step (collect): Requests specific information from the user and stores it in slots.

```
- collect: phone_number # slot name  
  description: "Request the user's phone number"
```

***Note:** The assistant will not proceed to the next step until the slot is filled.

Action Step (action): Executes a specific action, such as calling a custom action or sending a response.

```
- action: action_confirm_order
```

Key Components of a Flow

Set Slots Step (`set_slots`): Updates the values of slots based on business logic.

```
- set_slots:  
  | delivery_address: null
```

***Note:** You can set a slot to **null** to clear its value. Can set multiple slots

Call or Link Step (`call` or `link`): Invokes another Flow as part of the current process.

```
- set_slots:  
  | delivery_address: null # slot_name: value
```

Managing Slots After Flow Completion

Default Behavior:

- Slots set through `collect` and `set_slots` are reset when a Flow completes.

Persisting Slot Values:

- Use the `persisted_slots` attribute to specify which slots should retain their values after the Flow ends.

```
flows:
  flow_name:
    description: "Description of the flow"
    persisted_slots:
      - slot_name_1
      - slot_name_2
    steps:
      # Flow steps
```

***Note:** Only slots set through `collect` or `set_slots` should be added to `persisted_slots`. Slots set by custom actions are automatically retained unless explicitly reset.

What is the Domain?

- The domain defines the universe in which your assistant operates. It specifies the intents, entities, slots, responses, forms, and actions your bot should know.

Key Components of the Domain:

- **Intents:** User goals or purposes expressed in their inputs.
- **Entities:** Specific data extracted from user inputs.
- **Slots:** Variables that store information to be remembered throughout the conversation.
- **Responses:** Predefined messages the assistant can send to users.
- **Actions:** Operations the assistant can perform, including custom actions.

Defining the Domain:

- The domain is typically defined in a YAML file named `domain.yml`

```
slots:
  recipient:
    type: text
    mappings:
      - type: from_llm
  amount:
    type: float
    mappings:
      - type: from_llm

responses:
  utter_ask_recipient:
    - text: "Who would you like to send money to?"

  utter_ask_amount:
    - text: "How much money would you like to send?"
```

What are Custom Actions?

- Custom actions allow your assistant to execute arbitrary code, enabling operations like calling external APIs, querying databases, or performing calculations.

When to Use Custom Actions:

- Fetching real-time data from external sources.
- Performing complex business logic not handled by standard responses.
- Interacting with databases or other backend systems.

Implementing Custom Actions:

1. Create the Action Class:

- Define a Python class that inherits from `Action`.
- Implement the `name` method to specify the action's identifier.
- Implement the `run` method to execute the desired functionality.

2. Register the Action:

- List the action in the `actions` section of your `domain.yml`.

3. Run the Action Server: `rasa run actions`

```
# Minimal custom action code
class CheckRestaurantAvailability(Action):
    def name(self):
        return "check_restaurant_availability"

    def run(self, dispatcher, tracker, domain):
        # Example: call an API to see if there's availability
        has_availability = True
        # Return the result in a slot so the flow can branch deterministically
        return [SlotSet("has_availability", has_availability)]
```

- **Introduction to Rasa NLU:**

- What is Rasa NLU? (Natural Language Understanding)
- Role of Rasa NLU in extracting intents and entities from user messages.
- Overview of intents, entities, and training data.

- **Components of Rasa NLU:**

- Intents: Define what the user wants to do (e.g., book a flight).
- Entities: Extract relevant information (e.g., from_city, to_city, date).
- Training

Create NLU training data in `data/nlu.yml` file with intents like **greet**, **book_flight**, **flight_info**

```
version: "3.1"

nlu:
  - intent: greet
    examples: |
      - Hi
      - Hello
      - Good morning

  - intent: book_flight
    examples: |
      - I want to book a flight from [New York](from_city) to [London](to_city) on [tomorrow](date)
      - Book a flight to [Paris](to_city) from [Tokyo](from_city) on [next Monday](date)
```

Configuring NLU Pipeline:

- Explanation of `config.yml` with pipeline components:

```
language: en

pipeline:
  - name: WhitespaceTokenizer
  - name: RegexFeaturizer
  - name: CountVectorsFeaturizer
  - name: DIETClassifier
  - name: EntitySynonymMapper
```

Training and Testing NLU Model:

- Training model use: `rasa train nlu`
- Testing model use: `rasa shell nlu`

Checking intents and entities extraction from user messages

Practical Exercise

- Build and train a basic NLU model for flight booking
- Test with sentences like:
 - "Tôi muốn đặt vé máy bay từ Hà Nội đến Đà Lạt"
 - "Đặt vé từ Đà Nẵng tới Huế vào thứ hai"

From Rasa NLU to Rasa CALM

Previous Slides:

- You may have worked with **Rasa NLU pipelines** (e.g., intent classification, entity extraction using `config.yml`)

In This Practical Session:

- We are using **Rasa CALM**
 - Focused on **Flow-based dialogue**, not intent-based
 - **No NLU pipeline is required**

What is Rasa CALM?

- **Rasa CALM (Conversational AI with Language Models)** is an enterprise-grade conversational AI framework built on top of Rasa Open Source, providing advanced features for developing, deploying, and managing complex AI assistants.
- Unlike Rasa Open Source, Rasa CALM is specifically designed for enterprise use cases with enhanced tools and integrations.

Key Features of Rasa CALM

LLM Integration

- Seamless connection to **GPT-4, open-source, or custom fine-tuned models**
- Improves both **language understanding** and **response generation**

Flow-based Dialogue Management

- Define structured conversations using **flows**
- Supports:
 - Multi-turn dialogues
 - Conditional logic
 - Dynamic slot handling

Key Features of Rasa CAI M

Enterprise Search Capabilities

- Integrates with **FAISS, Milvus, Qdrant**
- Enables **context-aware responses** from large datasets

Customization & Fine-Tuning

- Train and fine-tune LLMs for **domain-specific tasks**
- Improve precision for **specialized use cases**

Enterprise-Ready Scalability

- Built to handle **high-load deployments**
- Includes tools for:
 - Monitoring
 - Maintenance
 - Performance optimization

Searching for Flights: Functional Requirements

The user wants to **search for available flights** based on:

- **Departure City** (`from_city`)
- **Destination City** (`to_city`)
- **Travel Date** (`travel_date`)

The assistant should:

- Collect the required information (slots) if missing
- Read flight data from a CSV file
- Filter and return a list of matching flights
- Return a fallback message if no flights are found

Searching for Flights: Expected Result

User: I want to find a flight.

Assistant: Where are you flying from?

User: From Paris.

Assistant: Where would you like to go?

User: Rome.

Assistant: What is your travel date?

User: April 15th.

Assistant: Available flights from Paris to Rome on April 15th:

- VN101 | 08:00 → 10:30 | Economy | \$150
- VN202 | 12:00 → 14:30 | Business | \$180

Step 1: Define Slots & Responses in domain.yml

- Define the **3 required slots**:
 - **from_city**: the departure location
 - **to_city**: the destination location
 - **travel_date**: the desired date of travel
- Create appropriate **responses**:
 - Ask for missing information (e.g., "Where are you flying from?")
 - Display fallback message if no flights match the query

Step 2: Create Flow `flows/search_flight.yml`

Build a flow to manage the dialogue steps:

- When the intent `search_flight` is detected, check if the required slots are filled
- If any slot is missing, ask for it using the corresponding `utter_ask_*` response
- Once all slots are filled, trigger a custom action (`action_search_flights`) to perform the actual search and return flight options

Step 3: Prepare Flight Data File

- Download a CSV file (`flight_data.csv`) or automatically create files containing flight information
- Required fields in the file should include:

`flight_code`: unique code for each flight

`departure_time`, `arrival_time`: time of departure and arrival

`from_city`, `to_city`: origin and destination cities

`available_seats`: number of remaining seats

`price`: ticket price

`class`: flight class (e.g., Economy, Business)

Step 4: Write Custom Action

In the `action_search_flights` custom action:

- Load the CSV file containing flight data
- Normalize and parse the `travel_date` into a consistent format (e.g., `YYYY-MM-DD`)
- Filter the dataset based on:
 - Matching `from_city` and `to_city` values
 - Flights that still have available seats (`available_seats > 0`)
- Format and return the matching flights in a readable message
- If no suitable flights are found, return a response informing the user accordingly

Testing Your Rasa Assistant

Step 1: Train Your Assistant

Ensure your assistant is trained with the latest data:

```
rasa train
```

Step 2: Launch Rasa Inspector

Use Rasa Inspector for an interactive, browser-based testing experience:

```
rasa inspect
```

This command opens a browser window featuring:

- **Chat Widget:** Engage in conversations with your assistant.
- **Flow View:** Visualize active flows and their current steps.
- **Stack View:** Monitor the sequence of activated flows.
- **Tracker State:** Inspect slot values, events, and conversation history.

Booking Flights: Functional Requirements

The user wants to **book a flight** by:

- Providing a **flight code** and **number of tickets**
- Entering personal information for **each passenger**, including:
 - Full name
 - ID number
 - Phone number

The system should:

- Loop through the number of passengers
- Collect and store each passenger's information
- Once all passengers are collected, confirm the booking and return a **booking code**

Booking Flights: Expected Result

```
User: I want to book a flight.
Assistant: Please provide the flight code.

User: VN101
Assistant: How many passengers are you booking for?

User: 2
Assistant: Please enter passenger #1's full name.

User: Alice Nguyen
Assistant: Enter their ID number.

User: A123456
Assistant: Enter their phone number.

User: 0912345678
Assistant: Please enter passenger #2's full name.
...

Assistant: Booking confirmed for flight VN101.
Your booking reference is BK20250425-001.
```


Booking Flights: Step-by-Step Implementation

Step 1: Define Required Slots in `domain.yml` and `flow.yml`

- You'll need the following slots:
 - `flight_code`: stores the flight identifier
 - `num_tickets`: stores the number of passengers to collect info for
 - `passenger_info`: list of passenger dictionaries (name, ID, phone)
 - `current_passenger_index`: to keep track of the current passenger being processed
 - Temporary slots for each passenger:
 - `passenger_name`
 - `passenger_id`
 - `passenger_phone`

Step 2: Collect Passenger Info with Sequential Custom Actions

- Use a **chain of custom actions** to collect and store passenger details in order:
 1. Ask for the **passenger's name** using the current index
 2. After the user replies, store the name and follow up with a prompt for **ID number**
 3. Store the ID and ask for the **phone number**
 4. Once the phone number is received:
 - Store the complete passenger info
 - Increase the passenger index
 - If more passengers remain → repeat the process
 - If finished → proceed to final booking step

Step 3: Store Passenger Data

- After each passenger's full data is collected:
 - Append it to the `passenger_info` slot
 - Clear temporary slots (`passenger_name`, `passenger_id`, etc.)
 - Update `current_passenger_index`
- When all passengers are collected:
 - Inform the user that passenger input is complete
 - Trigger a follow-up action (e.g. `action_book_selected_flight`) to finalize the booking

Step 4: Confirm Booking and Return Result

- Once all data is collected, the final action should:
 - Check if the flight exists and has enough seats
 - Confirm the booking
 - Generate a **booking reference code**
 - Return a success message with a summary

Check Booking Infor: Functional Requirements

The user wants to **review the details of a booking** by providing a **booking code**

The system should:

- Ask for the booking code if it's not already provided
- Look up the booking information (from database, CSV, or in-memory mock data)
- Return the relevant booking details such as:
 - Flight code
 - Passenger list
 - Departure & destination cities
 - Travel date
 - Number of passengers

Check Booking Infor: Expected Result

User: I want to check my booking.

Assistant: Please provide your booking code.

User: BK20250425-001

Assistant: Booking BK20250425-001:

- Flight: VN101
- Route: Paris → Rome
- Date: April 15th
- Passengers:

- Alice Nguyen	A123456	0912345678
- Bob Tran	B789123	0987654321

Check Booking Infor: Step-by-Step Implementation

Step 1: Define Slot & Responses in `domain.yml` and create flow `check_booking`

- Define the following slot:
 - `booking_code`: to capture the booking reference
- Define responses:
 - `utter_ask_booking_code`: ask user to provide the booking code
 - `utter_booking_not_found`: response if no matching booking is found
 - You may also define `utter_booking_details` or return message in custom action

Step 2: Implement Custom Action `action_check_booking`

- Retrieve the slot `booking_code`
- Check the booking data:
 - If stored **in memory** (e.g., domain slot or global variable), scan the list of bookings
 - If stored in **CSV file**, read the file and find the matching booking row
- Once a match is found:
 - Parse flight details: flight code, route, date
 - Parse passenger list (name, ID, phone)
 - Construct a summary message
- If no match is found, return `utter_booking_not_found`

Cancel Booking: Functional Requirements

- The user wants to **cancel a flight booking**
- The system should:
 - Ask for the **booking code** if it's not provided
 - Retrieve the booking details
 - Verify:
 - Whether the **ticket class allows cancellation** (e.g., Economy may not be refundable)
 - Whether the **current date is before the flight date**
 - If valid → cancel the booking and confirm
 - If invalid → explain why cancellation is not allowed

Cancel Booking: Expected Result

User: I want to cancel my flight.

Assistant: Please provide your booking code.

User: BK20250425-001

Assistant: Booking found. Checking eligibility...

Assistant: Your booking cannot be canceled because it is an Economy ticket.

Cancel Booking: Step-by-Step Implementation

Step 1: Define Slot & Responses in `domain.yml` and create flow `cancel_flight`

- Define:
 - `booking_code` slot
- Create responses:
 - `utter_ask_cancel_code`: prompt for booking code
 - `utter_cancellation_successful`: confirmation if cancel is valid
 - `utter_cancellation_denied`: if cancelation fails due to rules
 - Optionally: `utter_booking_not_found` if the code is invalid

Cancel Booking: Step-by-Step Implementation

Step 2: Logic in Custom Action `action_cancel_booking`

Inside the action:

- Retrieve `booking_code` from the tracker
- Find the booking record (from in-memory list or CSV file)
- Check these 2 cancellation rules:
 - 1. Class Type Rule**
 - Business, Premium → allowed to cancel
 - Economy → not allowed to cancel
 - 2. Flight Date Rule**
 - Only allow cancellation if today's date < travel date
 - (You can use `datetime.strptime()` and `datetime.now()` to compare)
- Based on the result:
 - If both conditions are met → confirm cancellation + optionally remove or mark booking as canceled
 - Else → inform user that cancellation is not allowed (specify reason)

A large, stylized graphic on the left side of the slide. It consists of a circular pattern of small dots in a lighter shade of red, arranged in a way that suggests a globe or a network. The word "HUST" is written in white, bold, sans-serif capital letters across the center of this graphic.

HUST

THANK YOU !